LeetCode Practice

14 Medium: Trie

Question, examples, then the six-phase interview approach with the brute force in phase 2

A split copy of the full All Questions book; every feature and link works the same

14
MEDIUM

Patterns (in order)

1. Trie (14)

How to use

- Tap any title in the Contents to jump to it. A topic name opens that topic's own divider page.
- Every page has < Prev | Contents | Next > at the top.
- Prev and Next move one question at a time; Contents returns here. Each topic starts on its own page.
- Select code and press Cmd/Ctrl+C to copy it.

Go to Contents >

Contents

14 questions | tap a title to jump to it

Medium questions (14)	Go >
Trie (14)	**Go >**
M 1. Word Break	LC 139
M 2. Implement Trie (Prefix Tree)	LC 208
M 3. Design Add and Search Words Data Structure	LC 211
M 4. Add Bold Tag in String	LC 616
M 5. Replace Words	LC 648
M 6. Top K Frequent Words	LC 692
M 7. Longest Word in Dictionary	LC 720
M 8. Number of Matching Subsequences	LC 792
M 9. Design File System	LC 1166
M 10. Remove Sub-Folders from the Filesystem	LC 1233
M 11. Search Suggestions System	LC 1268
M 12. Number of Distinct Substrings in a String	LC 1698
M 13. Words Within Two Edits of Dictionary	LC 2452
M 14. Find the Length of the Longest Common Prefix	LC 3043

Medium questions (14)

MEDIUM | TOPIC 1 OF 1

Trie

14 Medium questions start on the next page

Before this: this is the first topic
After this: this is the last topic

IN THIS TOPIC

1. Word Break LC 139
2. Implement Trie (Prefix Tree) LC 208
3. Design Add and Search Words Data Structure LC 211
4. Add Bold Tag in String LC 616
5. Replace Words LC 648
6. Top K Frequent Words LC 692
7. Longest Word in Dictionary LC 720
8. Number of Matching Subsequences LC 792
9. Design File System LC 1166
10. Remove Sub-Folders from the Filesystem LC 1233
11. Search Suggestions System LC 1268
12. Number of Distinct Substrings in a String LC 1698
13. Words Within Two Edits of Dictionary LC 2452
14. Find the Length of the Longest Common Prefix LC 3043

MEDIUM | Trie | LeetCode #139 | 1 of 14 | 1/14

Word Break

QUESTION

Given a string s and a dictionary of strings wordDict, return true if s can be segmented into a space-separated sequence of one or more dictionary words.

Note that the same word in the dictionary may be reused multiple times in the segmentation.

Example 1	
Input	`s = "leetcode", wordDict = ["leet","code"]`
Output	`true`
Explanation	*Return true because "leetcode" can be segmented as "leet code".*

Example 2	
Input	`s = "applepenapple", wordDict = ["apple","pen"]`
Output	`true`
Explanation	*Return true because "applepenapple" can be segmented as "apple pen apple". Note that you are allowed to reuse a dictionary word.*

Example 3	
Input	`s = "catsandog", wordDict = ["cats","dog","sand","and","cat"]`
Output	`false`

CONSTRAINTS

- 1 <= s.length <= 300
- 1 <= wordDict.length <= 1000
- 1 <= wordDict[i].length <= 20
- s and wordDict[i] consist of only lowercase English letters.
- All the strings of wordDict are unique.

INTERVIEW APPROACH · STAR-LC

PHASE 1 CLARIFY

Let me make sure I understand the problem correctly.

Given string s and a dictionary of words, return true if s can be segmented

into a sequence of dictionary words. Words can be reused. Right?"

"A few quick questions:

Can the empty string be segmented? Edge case — but s length >= 1 per constraints.

Same word can be used multiple times? Yes."

PHASE 1 CLARIFY

"Let me walk: s='leetcode', words=['leet','code']. 'leet'+'code' -> true ok

s='catsandog', words=['cats','dog','sand','and','cat'] -> false ok

PHASE 2 BRUTE FORCE

"Let me start with brute force to lock in the logic."

BRUTE FORCE · PSEUDO-CODE

```
def ways(i):
    if i is the end:
        return True
    for j in range(n):
        if s[i..j] is in the dictionary and ways(j):
            return True
    return False
```

TIME	SPACE
O(2^n)	O(n)

WHY IT IS SLOW

The same suffix is re-solved from many split points. A table where ok[i] means 'the suffix from i can be split' makes it O(n^2) (with substring checks) or better.

"Should I go ahead and optimize?"

PHASE 3 OPTIMIZE

The bottleneck is the repeated subproblem: can(i) is recomputed many times.

Bottom-up DP: dp[i] = True if s[:i] can be segmented.

Base: dp[0] = True (empty prefix).

Transition: dp[i] = any(dp[j] and s[j:i] in wordSet) for 0 <= j < i.

Optimise by only checking back max_word_len characters.

Time: O(n * max_word_len). Space: O(n)."

Key insight: dp[i] = True if any split s[:j]+s[j:i] works with dp[j] True and s[j:i] in the word set — filling left-to-right computes each cell from already-solved subproblems.

PHASE 4 CLEAN CODE

Now I'll implement the optimized approach with meaningful names.

```python
class _139_WordBreak:
    def wordBreak(self, s, wordDict):
        word_set = set(wordDict)
        max_len = max(len(w) for w in wordDict)
        n = len(s)
        dp = [False] * (n + 1)
        dp[0] = True
        for i in range(1, n + 1):
            for j in range(max(0, i - max_len), i):
                if dp[j] and s[j:i] in word_set:
                    dp[i] = True
                    break
        return dp[n]
```

PHASE 5 TEST & DEBUG

"Let me trace wordBreak('leetcode',['leet','code'])."

`dp[i] = can s[:i] be segmented. dp[0]=True (empty string).`

i=1('l'): no word ends here -> dp[1]=F.

i=2('le'): no -> F. i=3('lee'): no -> F.

i=4('leet'): check j=0..3: j=0 -> s[0:4]='leet' in words AND dp[0]=T -> dp[4]=T ok

i=5: 'l','le','lee','leet','leetc'. 'leet' breaks dp[4:5]='c'? not in words. j=1..4: no match. dp[5]=F.

i=6('leetco'): check. No single word 'code' starts at i-4=2. dp[6]=F.

i=7('leetcod'): dp[7]=F. i=8('leetcode'): j=4 -> s[4:8]='code' in words AND dp[4]=T -> dp[8]=T ok

Return True ok

PHASE 6 COMPLEXITY & FOLLOW-UP

TIME	SPACE
O(n * max_word_len) n positions × max_word_len substrings each	**O(n)**

Trie optimisation: insert all words into a trie, walk forward from each dp[j]=True.

Follow-up: Word Break II (#140) — return all valid sentences; backtrack from dp[n]=True.

Follow-up: if wordDict is huge but s is short, iterate over words instead of positions.

MEDIUM | Trie | LeetCode #208 | 2 of 14 | 2/14

Implement Trie (Prefix Tree)

QUESTION

A trie (pronounced as "try") or prefix tree is a tree data structure used to efficiently store and retrieve keys in a dataset of strings. There are various applications of this data structure, such as autocomplete and spellchecker.

Implement the Trie class:

CONSTRAINTS

- Trie() Initializes the trie object.
- void insert(String word) Inserts the string word into the trie.
- boolean search(String word) Returns true if the string word is in the trie (i.e., was inserted before), and false otherwise.
- boolean startsWith(String prefix) Returns true if there is a previously inserted string word that has the prefix prefix, and false otherwise.
- 1 <= word.length, prefix.length <= 2000
- word and prefix consist only of lowercase English letters.
- At most 3 * 10^4 calls in total will be made to insert, search, and startsWith.

INTERVIEW APPROACH · STAR-LC

PHASE 1 CLARIFY

Let me make sure I understand the problem correctly.

Implement a Trie with insert(word), search(word)->bool (exact match),

and startsWith(prefix)->bool. Lowercase letters only. Right?"

"A few quick questions:

search('app') should return false even after inserting 'apple'? Yes — exact match only.

startsWith('app') should return true? Yes — any inserted word with that prefix."

"Let me walk: insert('apple'). search('apple')->T. search('app')->F.

startsWith('app')->T. insert('app'). search('app')->T ok

PHASE 2 BRUTE FORCE

"Let me start with brute force to lock in the logic."

BRUTE FORCE · PSEUDO-CODE

```
words = every inserted word in a list
def search(w):
    scan the list for w
def startsWith(p):
    scan the list for any word starting with p
```

TIME	SPACE
O(N * L) per query	O(N * L)

WHY IT IS SLOW

PHASE 2 BRUTE FORCE

Every query scans all N words. A trie follows one edge per character, so insert, search and startsWith are O(L).

"Should I go ahead and optimize?"

PHASE 3 OPTIMIZE

The bottleneck is the O(n*m) prefix scan.

True trie: each node has up to 26 children and an is_end flag.

insert: create nodes along the path, mark is_end at the last node.

search: walk path, return is_end of the final node (or False if path missing).

startsWith: walk path, return True if the path exists completely.

Time: O(m) per operation where m = word/prefix length.

Space: O(total characters * 26).

PHASE 4 CLEAN CODE

Now I'll implement the optimized approach with meaningful names.

```python
class _208_ImplementTrie:
    def __init__(self):
        self._children = {}
        self._is_end = False

    def insert(self, word):
        node = self
        for ch in word:
            if ch not in node._children:
                node._children[ch] = _208_ImplementTrie()
            node = node._children[ch]
        node._is_end = True

    def search(self, word):
        node = self
        for ch in word:
            if ch not in node._children:
                return False
            node = node._children[ch]
        return node._is_end

    def startsWith(self, prefix):
        node = self
        for ch in prefix:
            if ch not in node._children:
                return False
            node = node._children[ch]
        return True
```

PHASE 5 TEST & DEBUG

"Let me trace the trie navigation (prefix matches) and how it decides the answer."

insert('apple'): root->a->p->p->l->e(is_end=True).

search('apple'): walk path -> is_end=True ok.

search('app'): walk a->p->p -> is_end=False -> False ok.

startsWith('app'): walk a->p->p -> path exists -> True ok.

PHASE 5 TEST & DEBUG

```
insert('app'): node at second p gets is_end=True.
  search('app')->True ok.
```

Using dict children is memory-efficient for sparse tries vs a fixed 26-slot array

Trace: root=[1,2,3] -> left subtree (2) checked then right (3); both children of root compared.

PHASE 6 COMPLEXITY & FOLLOW-UP

TIME	SPACE
O(m)	**O(n*m)**
per op: walk at most m nodes	n words of avg length m

Dict children vs array[26]: dict saves space for sparse alphabets.

Follow-up: delete a word — mark is_end=False; prune childless nodes bottom-up.

Follow-up: Word Search II (#212) — DFS the board against a trie of all words.

MEDIUM | Trie | LeetCode #211 | 3 of 14 | 3/14

Design Add and Search Words Data Structure

QUESTION

Design a data structure that supports adding new words and finding if a string matches any previously added string.

Implement the WordDictionary class:

CONSTRAINTS

- WordDictionary() Initializes the object.
- void addWord(word) Adds word to the data structure, it can be matched later.
- bool search(word) Returns true if there is any string in the data structure that matches word or false otherwise. word may contain dots '.' where dots can be matched with any letter.
- 1 <= word.length <= 25
- word in addWord consists of lowercase English letters.
- word in search consist of '.' or lowercase English letters.
- There will be at most 2 dots in word for search queries.
- At most 10^4 calls will be made to addWord and search.

INTERVIEW APPROACH · STAR-LC

PHASE 1 CLARIFY

Let me make sure I understand the problem correctly.

Design a data structure with addWord(word) and search(word).

search supports '.' as a wildcard matching any single letter. Right?"

"A few quick questions:

Can the pattern have multiple dots? Yes — up to 2 per problem constraints.

search('.') should match any one-character word? Yes."

"Let me walk: addWord('bad','dad','mad'). search('.ad')->true. search('b..')->true ok

PHASE 2 BRUTE FORCE

"Let me start with brute force to lock in the logic."

BRUTE FORCE · PSEUDO-CODE

```
words = a list
def search(pattern):
    compare it with every stored word, treating
        '.' as matching any letter
```

TIME	SPACE
O(N * L) per search	O(N * L)

WHY IT IS SLOW

Every search scans all N words. Store words in a trie and search by DFS, branching into all children only at a '.'.

PHASE 2 BRUTE FORCE

"Should I go ahead and optimize?"

PHASE 3 OPTIMIZE

The bottleneck is scanning all words on each search.

Trie with DFS for wildcards:

addWord: standard trie insert O(m).

search: walk trie; on '.' try ALL children recursively; on a letter take the matching child.

Prune any branch that leads to a missing character.

Time: addWord O(m). search O(m) best, O(26^dots * m) worst with dots.

Space: O(total chars)."

Time: O(log n) — binary search on answer space.

Key insight: the Trie prunes character-by-character — a '.' wildcard fans out only to existing children (not all 26 letters), so deep mismatches cut entire subtrees cheaply.

PHASE 4 CLEAN CODE

Now I'll implement the optimized approach with meaningful names.

```
class _211_AddSearchWords:
    def __init__(self):
        self._children = {}
        self._is_end = False

    def addWord(self, word):
        node = self
        for ch in word:
            if ch not in node._children:
                node._children[ch] = _211_AddSearchWords()
            node = node._children[ch]
        node._is_end = True

    def search(self, word):
        def dfs(node, i):
            if i == len(word):
                return node._is_end
            ch = word[i]
            if ch == '.':
                return any(dfs(child, i + 1) for child in
node._children.values())
            if ch not in node._children:
                return False
            return dfs(node._children[ch], i + 1)
        return dfs(self, 0)
```

PHASE 5 TEST & DEBUG

"Let me trace the trie navigation (prefix matches) and how it decides the answer."

addWord('bad','dad','mad'): b->a->d(end), d->a->d(end), m->a->d(end).

search('.ad'): dfs(root,0): ch='.', try b,d,m -> dfs(b_node,1): 'a'->dfs(a_node,2): 'd'->is_end=T -> True ok

search('b..'): dfs(root,0): 'b'->dfs(b,1): '.'->try 'a'->dfs(a,2): '.'->try 'd'->is_end=T -> True ok

PHASE 5 TEST & DEBUG

```
search('pad'): 'p' not in children -> False ok
```

DFS correctly backtracks when a '.' branch leads to a dead end

Trace: nums=[1,3,5,7,9], target=5 -> low=0,high=4,mid=2 -> nums[2]=5 == target -> return 2

PHASE 6 COMPLEXITY & FOLLOW-UP

addWord O(m). search O(m) for no dots; O(26^k * m) worst with k dots.

In practice dots are rare, so average is close to O(m).

Follow-up: if patterns are very dot-heavy, cache results at (node, pattern_index) pairs.

Follow-up: support '*' (zero or more chars) — extended DFS with skip-0 and skip-1+ logic."

Space: O(n×|?|) — trie stores n characters with branching factor |alphabet|.

IN YOUR APP | Expose | Editorial | Community

MEDIUM | Trie | LeetCode #616 | 4 of 14 | 4/14

Add Bold Tag in String

QUESTION

You are given a string s and an array of strings words.

You should add a closed pair of bold tag and to wrap the substrings in s that exist in words.

CONSTRAINTS

- If two such substrings overlap, you should wrap them together with only one pair of closed bold-tag.
- If two substrings wrapped by bold tags are consecutive, you should combine them.

Return s after adding the bold tags.

Example 1

Input	`s = "abcxyz123", words = ["abc","123"]`
Output	`"<b>abc</b>xyz<b>123</b>"`
Explanation	*The two strings of words are substrings of s as following: " abc xyz 123 ". We add before each substring and after each substring.*

Example 2

Input	`s = "aaabbb", words = ["aa","b"]`
Output	`"<b>aaabbb</b>"`
Explanation	*"aa" appears as a substring two times: " aa abbb" and "a aa bbb". "b" appears as a substring three times: "aaa b bb", "aaab b b", and "aaabb b ". We add before each substring and after each substring: "aaabbb". Since the first two 's overlap, we merge them: "aaabbb". Since now the four 's are consecutive, we merge them: "aaabbb".*

- 1 <= s.length <= 1000
- 0 <= words.length <= 100
- 1 <= words[i].length <= 1000
- s and words[i] consist of English letters and digits.
- All the values of words are unique.
- Note: This question is the same as 758. Bold Words in String.

INTERVIEW APPROACH · STAR-LC

PHASE 1 CLARIFY

Let me make sure I understand the problem correctly.

Wrap all substrings of s that appear in the words list with tags.

Overlapping or adjacent bold regions should be merged into one. Right?"

"A few quick questions:

A word can appear multiple times in s — all occurrences get bolded? Yes.

Overlapping matches merge into one ... block? Yes."

"Let me walk: s='abcxyz123', words=['abc','123'] -> 'abcxyz123' ok

PHASE 2 BRUTE FORCE

"Let me start with brute force to lock in the logic."

BRUTE FORCE · PSEUDO-CODE

```
for position in positions:
    # in s and every word
    if the word starts there:
        mark its characters bold
    wrap each maximal bold run in tags
```

TIME	SPACE
O(n * W * L)	O(n)

WHY IT IS SLOW

Tests every word at every position. A trie of the words, or an Aho-Corasick automaton, finds all matches in one scan; the bold marks are then merged as intervals.

"Should I go ahead and optimize?"

PHASE 3 OPTIMIZE

The bottleneck is the O(n * m * L) find loop.

Trie optimization: insert all words into a trie, then scan s once.

At each position, walk the trie to find all word matches starting there.

Mark the corresponding range in the bold array.

O(n * max_L) scan.

But the brute with bool array is already clean — let's refine it using intervals."

Time: O(m) per operation — trie traversal by word length m.

PHASE 4 CLEAN CODE

Now I'll implement the optimized approach with meaningful names.

```python
class _616_AddBoldTag:
    def addBoldTag(self, s, words):
        n = len(s)
        bold = [False] * n
        for word in words:
            wl = len(word)
            for i in range(n - wl + 1):
                if s[i:i+wl] == word:
                    for j in range(i, i + wl):
                        bold[j] = True
        result = []; i = 0
        while i < n:
            if bold[i]:
                result.append('<b>')
                while i < n and bold[i]:
                    result.append(s[i]); i += 1
                result.append('</b>')
            else:
                result.append(s[i]); i += 1
        return ''.join(result)
```

PHASE 5 TEST & DEBUG

"Let me trace the trie navigation (prefix matches) and how it decides the answer."

s='aaabbb', words=['aa','b']:

'aa': matches at 0,1 -> bold[0..2]=True.

'b': matches at 3,4,5 -> bold[3..5]=True. All bold -> 'aaabbb' ok

s='abcxyz123', words=['abc','123']:

bold[0..2]=True, bold[6..8]=True. -> 'abcxyz123' ok

Adjacent True ranges merge naturally since the while loop consumes all

Trace: s="abcxyz123", words=["abc","123"] -> mark [0,3) and [6,9) as bold -> output: "abcxyz123" ok. Merged: if bold regions overlap, extend rather than close and reopen.

PHASE 6 COMPLEXITY & FOLLOW-UP

TIME	SPACE
O(n * m * L)	**O(n)**
n positions × m words × L substring check	bold array

Trie reduces substring matching to O(n * max_L) — worth it when m is large.

Follow-up: Bold Words in String (#758) — identical problem, same solution.

Follow-up: Merge Intervals (#56) — if instead of a bool array we track intervals directly.

MEDIUM | Trie | LeetCode #648 | 5 of 14 | 5/14

Replace Words

QUESTION

In English, we have a concept called root, which can be followed by some other word to form another longer word - let's call this word derivative. For example, when the root "help" is followed by the word "ful", we can form a derivative "helpful".

Given a dictionary consisting of many roots and a sentence consisting of words separated by spaces, replace all the derivatives in the sentence with the root forming it. If a derivative can be replaced by more than one root, replace it with the root that has the shortest length.

Return the sentence after the replacement.

Example 1

Input	`dictionary = ["cat","bat","rat"], sentence = "the cattle was rattled by the battery"`
Output	`"the cat was rat by the bat"`

Example 2

Input	`dictionary = ["a","b","c"], sentence = "aadsfasf absbs bbab cadsfafs"`
Output	`"a a b c"`

CONSTRAINTS

- 1 <= dictionary.length <= 1000
- 1 <= dictionary[i].length <= 100
- dictionary[i] consists of only lower-case letters.
- 1 <= sentence.length <= 10^6
- sentence consists of only lower-case letters and spaces.
- The number of words in sentence is in the range [1, 1000]
- The length of each word in sentence is in the range [1, 1000]
- Every two consecutive words in sentence will be separated by exactly one space.
- sentence does not have leading or trailing spaces.

INTERVIEW APPROACH · STAR-LC

PHASE 1 CLARIFY

Let me make sure I understand the problem correctly.

Replace each word in sentence with the shortest root in dictionary that is a prefix of it. Right?"

"A few quick questions:

If no root is a prefix, keep the word unchanged? Yes."

"Let me walk: dictionary=['cat','bat','rat'], sentence='the cattle was rattled by the battery'

-> 'the cat was rat by the bat' ok"

PHASE 1 CLARIFY

Constraints: dictionary up to 1000 roots, each up to length 100; sentence words up to length 1000; all lowercase.

PHASE 2 BRUTE FORCE

"Let me start with brute force to lock in the logic."

BRUTE FORCE · PSEUDO-CODE

```
for word in sentence:
    check it against every root and keep the
        shortest root that is a prefix
```

TIME	SPACE
O(W * R * L)	O(1)

WHY IT IS SLOW

Compares each word with every root. Put the roots in a trie and walk each word down it until a root ends: O(total characters).

"Should I go ahead and optimize?"

PHASE 3 OPTIMIZE

"Trie of dictionary roots — O(total_chars). Walk-through of how replacement works:"

Step 1 — Build Trie from dictionary:

For each root word: insert char by char into trie. Mark end of word.

Step 2 — For each word in sentence:

Traverse the Trie char by char.

If we hit an end-of-word marker: STOP — this is the shortest matching root. Use it.

If we exhaust the trie without matching: keep the original word.

WHY Trie finds the SHORTEST root efficiently:

We traverse character by character and stop at the FIRST end-of-word marker.

Longer roots would continue deeper in the trie; we never reach them.

Alternative: sort roots by length and check each word against all roots -> O(words * roots * L).

Trie is faster: O(word_length) per word, regardless of how many roots exist.

Time: O(dict_chars + sentence_chars). Space: O(dict_chars).

PHASE 4 CLEAN CODE

Now I'll implement the optimized approach with meaningful names.

```python
class _648_ReplaceWords:
    def replaceWords(self, dictionary, sentence):
        root_set = set(dictionary)
        def find_root(word):
            for i in range(1, len(word) + 1):
                if word[:i] in root_set:
                    return word[:i]
            return word
```

```
        return ' '.join(find_root(w) for w in sentence.spl
it())
```

PHASE 5 TEST & DEBUG

"Let me trace replaceWords(['cat','bat','rat'], 'the cattle was rattled by the battery')."

Build trie from roots: cat, bat, rat

For each word in sentence: find shortest trie prefix

'the': not in trie -> keep 'the'

'cattle': trie search: c->a->t -> 'cat' is a word in trie -> replace with 'cat'

'was': not in trie -> keep 'was'

'rattled': r->a->t -> 'rat' found -> replace with 'rat'

'by': keep. 'the': keep. 'battery': b->a->t -> 'bat' found -> replace with 'bat'

Result: 'the cat was rat by the bat' ok

WHY trie: O(sum of root lengths) build, O(word_length) lookup per word.

Trace: dict=["cat","bat","rat"], sentence="the cattle was rattled by the battery" -> "cattle" has prefix "cat" ok -> "cat"; "rattled" has prefix "rat" ok -> "rat"; "battery" has prefix "bat" ok -> "bat" -> "the cat was rat by the bat" ok.

PHASE 6 COMPLEXITY & FOLLOW-UP

TIME	SPACE
O(sum of word_len * root_len) per word. Trie: O(total_chars)	**O(n×|?|)** trie stores n characters with branching factor |alphabet|

Follow-up: if roots can be very long, Trie is significantly faster.

Follow-up: Longest Word in Dictionary (#720) — similar prefix structure."

IN YOUR APP | Expose | Editorial | Community

MEDIUM | Trie | LeetCode #692 | 6 of 14 | 6/14

Top K Frequent Words

QUESTION

Given an array of strings words and an integer k, return the k most frequent strings.

Return the answer sorted by the frequency from highest to lowest. Sort the words with the same frequency by their lexicographical order.

Example 1

Input	`words = ["i","love","leetcode","i","love","coding"], k = 2`
Output	`["i","love"]`
Explanation	*"i" and "love" are the two most frequent words. Note that "i" comes before "love" due to a lower alphabetical order.*

Example 2

Input	`words = ["the","day","is","sunny","the","the","the","sunny","is","is"], k = 4`
Output	`["the","is","sunny","day"]`
Explanation	*"the", "is", "sunny" and "day" are the four most frequent words, with the number of occurrence being 4, 3, 2 and 1 respectively.*

CONSTRAINTS

- 1 <= words.length <= 500
- 1 <= words[i].length <= 10
- words[i] consists of lowercase English letters.
- k is in the range [1, The number of unique words[i]]
- Follow-up: Could you solve it in O(n log(k)) time and O(n) extra space?

INTERVIEW APPROACH · STAR-LC

PHASE 1 CLARIFY

Let me make sure I understand the problem correctly.

Return the k most frequent words, sorted by frequency descending.

Ties broken alphabetically ascending. Right?"

"A few quick questions:

k is always valid (<= unique words)? Yes. Words are lowercase only?"

"Let me walk: words=['i','love','leetcode','i','love','coding'], k=2.

`i(2)>love(2) alphabetically: i < love -> [i,love] ok`

PHASE 2 BRUTE FORCE

"Let me start with brute force to lock in the logic."

BRUTE FORCE · PSEUDO-CODE

```
count the words
sort the distinct words by (count descending, word)
return the first k
```

TIME	SPACE
O(d log d)	O(d)

WHY IT IS SLOW

Sorts all d distinct words to keep k. A heap of size k is O(d log k).

"Should I go ahead and optimize?"

PHASE 3 OPTIMIZE

The bottleneck is sorting all unique words.

Min-heap of size k with a custom comparator handles this in O(n log k):

Store (-freq, word) — heap evicts the least desirable (smallest freq, then lex-largest).

This keeps the k best candidates at all times.

Time: O(n log k). Space: O(k)."

Key insight: a min-heap of size k with (-freq, word) ordering evicts the least desirable word when full — the final k-element heap IS the answer, avoiding a full O(n log n) sort.

PHASE 4 CLEAN CODE

Now I'll implement the optimized approach with meaningful names.

```python
import heapq
class _692_TopKFrequentWords:
    def topKFrequent(self, words, k):
        from collections import Counter
        count = Counter(words)
        heap = []
        for word, freq in count.items():
            heapq.heappush(heap, (-freq, word))
        return [heapq.heappop(heap)[1] for _ in range(k)]
```

PHASE 5 TEST & DEBUG

"Let me trace the trie navigation (prefix matches) and how it decides the answer."

words=['i','love','leetcode','i','love','coding'], k=2:

count={i:2,love:2,leetcode:1,coding:1}.

heap: [(-2,'i'),(-2,'love'),(-1,'coding'),(-1,'leetcode')] (min-heap by -freq then word).

pop1: (-2,'i'). pop2: (-2,'love'). -> ['i','love'] ok

Negating freq makes Python's min-heap act as a max-heap on frequency

PHASE 5 TEST & DEBUG

heap=[(1,'dad'),(1,'sad'),(2,'love'),(2,'leet'),(3,'i')]. Pop k=1: (1,'dad') then (1,'love')? Sort by (-count,word): [('i',3),('love',2),('leet',2),...]. Top 2=['i','love'] ok

PHASE 6 COMPLEXITY & FOLLOW-UP

Heap O(n log k). Sort O(n log n). Heap wins when k << n.

Tuple comparison (-freq, word) handles ties alphabetically for free in Python.

Follow-up: Top K Frequent Elements (#347) — integers, no tie-breaking, bucket sort O(n).

Follow-up: streaming top-k words — heapq.nsmallest after each word arrives."

Space: O(n×|?|) — trie stores n characters with branching factor |alphabet|.

IN YOUR APP | Expose | Editorial | Community

MEDIUM | Trie | LeetCode #720 | 7 of 14 | 7/14

Longest Word in Dictionary

QUESTION

Given an array of strings words representing an English Dictionary, return the longest word in words that can be built one character at a time by other words in words.

If there is more than one possible answer, return the longest word with the smallest lexicographical order. If there is no answer, return the empty string.

Note that the word should be built from left to right with each additional character being added to the end of a previous word.

Example 1

Input	`words = ["w","wo","wor","worl","world"]`
Output	`"world"`
Explanation	*The word "world" can be built one character at a time by "w", "wo", "wor", and "worl".*

Example 2

Input	`words = ["a","banana","app","appl","ap","apply","apple"]`
Output	`"apple"`
Explanation	*Both "apply" and "apple" can be built from other words in the dictionary. However, "apple" is lexicographically smaller than "apply".*

CONSTRAINTS

- 1 <= words.length <= 1000
- 1 <= words[i].length <= 30
- words[i] consists of lowercase English letters.

INTERVIEW APPROACH · STAR-LC

PHASE 1 CLARIFY

Let me make sure I understand the problem correctly.

Return the longest word where every prefix (excluding the full word) also exists in the list.

Ties: return the lexicographically smallest. Return '' if none. Right?"

"A few quick questions:

A single-letter word is always valid since it has no proper prefixes to check? Yes.

Words are all lowercase? Yes."

"Let me walk: words=['w','wo','wor','worl','world'] -> 'world' (all prefixes present) ok

words=['a','banana','app','appl','ap','apply','apple'] -> 'apple' (lex smaller than 'apply') ok

PHASE 2 BRUTE FORCE

"Let me start with brute force to lock in the logic."

BRUTE FORCE · PSEUDO-CODE

```
for word in words:
    check that each of its prefixes is in the
        word set
    best = max(best, candidate)
```

TIME	SPACE
O(n * L^2)	O(n * L)

WHY IT IS SLOW

Builds and hashes every prefix of every word. Sort the words and add a word only when its prefix without the last letter is already built (or walk a trie): O(total characters) after the sort.

"Should I go ahead and optimize?"

PHASE 3 OPTIMIZE

The bottleneck is the O(L²) prefix check per word.

Trie approach: insert all words, then BFS/DFS visiting only nodes marked is_end=True.

The deepest reachable node via only-end-marked paths gives the longest valid word.

Time: O(n*L) to build trie + O(n*L) to search. Space: O(n*L)."

Key insight: a BFS/DFS that visits only nodes marked is_end=True explores only "buildable" words — the deepest such node is the answer without checking all prefixes.

PHASE 4 CLEAN CODE

Now I'll implement the optimized approach with meaningful names.

```python
class _720_LongestWordInDictionary:
    def longestWord(self, words):
        word_set = set(words)
        result = ''
        for word in words:
            if all(word[:i] in word_set for i in range(1, len(word))):
                if len(word) > len(result) or (len(word) == len(result) and word < result):
                    result = word
        return result
```

PHASE 5 TEST & DEBUG

"Let me trace the trie navigation (prefix matches) and how it decides the answer."

words=['w','wo','wor','worl','world']:

'world': 'w'ok,'wo'ok,'wor'ok,'worl'ok -> len=5 > len('')=0 -> result='world'. -> 'world' ok

words=['a','banana','app','appl','ap','apply','apple']:

'apple': 'a','ap','app','appl' all in set ok. len=5. result='apple'.

PHASE 5 TEST & DEBUG

'apply': all prefixes ok. len=5=5, 'apply'>'apple' -> keep 'apple'. ok

The len/lex comparison handles ties correctly without sorting

PHASE 6 COMPLEXITY & FOLLOW-UP

TIME	SPACE
O(n * L²)	**O(n*L)**
n words × L prefixes × L per set lookup	for the set

Trie reduces prefix lookup to O(1) per char, bringing it to O(n*L).

Follow-up: if words stream in one at a time, maintain a trie with is_end flags.

Follow-up: allow at most one missing prefix — extend the BFS to allow one 'gap'.

IN YOUR APP | Expose | Editorial | Community

MEDIUM | Trie | LeetCode #792 | 8 of 14 | 8/14

Number of Matching Subsequences

QUESTION

Given a string s and an array of strings words, return the number of words[i] that is a subsequence of s.

A subsequence of a string is a new string generated from the original string with some characters (can be none) deleted without changing the relative order of the remaining characters.

CONSTRAINTS

- For example, "ace" is a subsequence of "abcde".

Example 1	
Input	`s = "abcde", words = ["a","bb","acd","ace"]`
Output	3
Explanation	*There are three strings in words that are a subsequence of s: "a", "acd", "ace".*

Example 2	
Input	`s = "dsahjpjauf", words = ["ahjpjau","ja","ahbwzgqnuk","tnmlanowax"]`
Output	2

- 1 <= s.length <= 5 * 10^4
- 1 <= words.length <= 5000
- 1 <= words[i].length <= 50
- s and words[i] consist of only lowercase English letters.

INTERVIEW APPROACH · STAR-LC

PHASE 1 CLARIFY

Let me make sure I understand the problem correctly.

Given a string s and list of words, count how many words are subsequences of s. Right?"

"A few quick questions:

A word appears in multiple places — count it once per occurrence in words list? Yes.

Empty string is a subsequence of everything? Yes."

"Let me walk: s='abcde', words=['a','bb','acd','ace'] -> 3 ('a','acd','ace') ok

PHASE 2 BRUTE FORCE

"Let me start with brute force to lock in the logic."

BRUTE FORCE · PSEUDO-CODE

```
for word in words:
    walk through s with a pointer into the word,
        advancing on a match
    count the words fully matched
```

PHASE 2 BRUTE FORCE

TIME	SPACE
O(W * |s|)	O(1)

WHY IT IS SLOW

Rescans s for every word. Bucket the words by their next needed letter and scan s once, moving each word to the bucket of its following letter: O(|s| + total word length).

"Should I go ahead and optimize?"

PHASE 3 OPTIMIZE

*"Group words by first-unmatched character into buckets — O(n * L). Walk-through:"*

For each word in words: bucket[word[0]].append((word, 0)). (char, index pointer)

For each char c in s:

`advance = []. next_bucket = defaultdict(list).`

For (word, ptr) in bucket[c]:

ptr++.

If ptr == len(word): count++. <- word fully matched

Else: next_bucket[word[ptr]].append((word, ptr)). <- waiting for next char

`bucket = next_bucket.`

WHY buckets: instead of checking all words at each s[i], only process words whose

current character matches s[i]. Distributes work across s.

Alternative: binary search + sorting (O(n L log n)).

Time: O(n * L) amortized. Space: O(n * L).

PHASE 4 CLEAN CODE

Now I'll implement the optimized approach with meaningful names.

```python
from collections import defaultdict
class _792_NumberMatchingSubseqs:
    def numMatchingSubseq(self, s, words):
        buckets = defaultdict(list)
        for word in words:
            buckets[word[0]].append(iter(word[1:]))
        count = 0
        for ch in s:
            waiting = buckets.pop(ch, [])
            for it in waiting:
                nxt = next(it, None)
                if nxt is None:
                    count += 1
                else:
                    buckets[nxt].append(it)
        return count
```

PHASE 5 TEST & DEBUG

"Let me trace the trie navigation (prefix matches) and how it decides the answer."

s='abcde', words=['a','bb','acd','ace']:

buckets: a->[iter(''),iter('cd'),iter('ce')], b->[iter('b')].

ch='a': move iters. iter('')->None(count=1). iter('cd')->'c'->buckets[c]. iter('ce')->'c'->buckets[c].

ch='b': move iter('b')->'b'? No, next of iter('b') after consuming 'b'=None? wait iter('b') starts at 'b'.

Hmm: words=['bb'] -> iter('b'). ch='a': no 'b' in waiting['a']? Correct, 'bb' starts with 'b'.

ch='b': waiting=[iter('b')]. next(iter('b'))='b'->buckets['b'].append(iter_remaining='').

ch='c','d','e': eventually all placed. -> count=3 ok

Iterators advance per character; iterator exhausted means word matched

PHASE 6 COMPLEXITY & FOLLOW-UP

TIME	SPACE
O(|s|+total_len)	**O(total_len)**
one pass through s + each word char processed once	all iterators at any time

Follow-up: if s changes per query, precompute next occurrence maps.

Follow-up: Is Subsequence (#392) — single word; O(|s|) two-pointer approach.

IN YOUR APP | Expose | Editorial | Community

MEDIUM | Trie | LeetCode #1166 | 9 of 14 | 9/14

Design File System

QUESTION

You are asked to design a file system that allows you to create new paths and associate them with different values.

The format of a path is one or more concatenated strings of the form: / followed by one or more lowercase English letters. For example, "/leetcode" and "/leetcode/problems" are valid paths while an empty string "" and "/" are not.

Implement the FileSystem class:

CONSTRAINTS

- bool createPath(string path, int value) Creates a new path and associates a value to it if possible and returns true. Returns false if the path already exists or its parent path doesn't exist.
- int get(string path) Returns the value associated with path or returns -1 if the path doesn't exist.

Example 1

Input	`["FileSystem","createPath","get"]` `[[],["/a",1],["/a"]]`
Output	`[null,true,1]`
Explanation	*FileSystem fileSystem = new FileSystem(); fileSystem.createPath("/a", 1); // return true fileSystem.get("/a"); // return 1*

Example 2

Input	`["FileSystem","createPath","createPath","get","createPath","get"]` `[[],["/leet",1],["/leet/code",2],["/leet/code"],["/c/d",1],["/c"]]`
Output	`[null,true,true,2,false,-1]`
Explanation	*FileSystem fileSystem = new FileSystem(); fileSystem.createPath("/leet", 1); // return true fileSystem.createPath("/leet/code", 2); // return true fileSystem.get("/leet/code"); // return 2 fileSystem.createPath("/c/d", 1); // return false because the parent path "/c" doesn't exist. fileSystem.get("/c"); // return -1 because this path doesn't exist.*

- 2 <= path.length <= 100
- 1 <= value <= 10^9
- Each path is valid and consists of lowercase English letters and '/'.
- At most 10^4 calls in total will be made to createPath and get.

INTERVIEW APPROACH · STAR-LC

PHASE 1 CLARIFY

Let me make sure I understand the problem correctly.

Design a file system with createPath(path, value)->bool and get(path)->int.

createPath creates a path with a value; parent must exist.

Returns false if path exists or parent doesn't exist.

get returns value or -1. Right?"

"A few quick questions:

Root '/' is always implicitly there? Yes. Paths start with '/'? Yes."

"Let me walk: createPath('/a',1)->T. createPath('/a/b',2)->T. get('/a')->1.

`createPath('/c/d',1)->F (no '/c'). get('/c')->-1 ok`

PHASE 2 BRUTE FORCE

"Let me start with brute force to lock in the logic."

BRUTE FORCE · PSEUDO-CODE

```
keep every created path in a list
createPath checks the parent by scanning the list
get scans for the path
```

TIME	SPACE
O(n) per operation	**O(n)**

WHY IT IS SLOW

Every call scans all paths. A hash map from path to value, with the parent found by cutting at the last '/', makes both operations O(path length).

"Should I go ahead and optimize?"

PHASE 3 OPTIMIZE

The bottleneck is the rfind scan for the parent path.

Trie where each node maps component names to child nodes and stores a value.

createPath: split on '/', walk/create nodes, check parent validity.

get: walk trie, return node value.

Time: O(L) per op (L = path length). Space: O(total unique path components)."

Time: O(m) per operation — trie traversal by word length m.

PHASE 4 CLEAN CODE

Now I'll implement the optimized approach with meaningful names.

```python
class _1166_DesignFileSystem:
    def __init__(self):
        self._store = {}

    def createPath(self, path, value):
```

```
        parts = path.split('/')
        parent = '/'.join(parts[:-1])
        if parent and parent not in self._store:
            return False
        if path in self._store:
            return False
        self._store[path] = value
        return True

    def get(self, path):
        return self._store.get(path, -1)
```

PHASE 5 TEST & DEBUG

"Let me trace the trie navigation (prefix matches) and how it decides the answer."

createPath('/a',1): parent=''. '' not in store? '' is falsy -> skip parent check.

path '/a' not in store -> store['/a']=1 -> True ok

createPath('/a/b',2): parent='/a'. '/a' in store ok. '/a/b' not in store -> store['/a/b']=2 ok

get('/a')->1 ok. createPath('/c/d',1): parent='/c'. '/c' not in store -> False ok

The 'if parent and ...' handles the root case cleanly

PHASE 6 COMPLEXITY & FOLLOW-UP

Both ops O(L): dict lookup + parent extraction.

Space O(total unique paths). No prefix sharing needed since we store full paths.

True trie would share prefixes but adds complexity — dict is simpler and equally fast here.

Follow-up: delete(path) — remove from dict; handle subtrees by scanning for prefix matches.

Follow-up: list directory — return keys that start with path+'/' and have no further '/'.

MEDIUM | Trie | LeetCode #1233 | 10 of 14 | 10/14

Remove Sub-Folders from the Filesystem

QUESTION

Given a list of folders folder, return the folders after removing all sub-folders in those folders. You may return the answer in any order.

If a folder[i] is located within another folder[j], it is called a sub-folder of it. A sub-folder of folder[j] must start with folder[j], followed by a "/". For example, "/a/b" is a sub-folder of "/a", but "/b" is not a sub-folder of "/a/b/c".

The format of a path is one or more concatenated strings of the form: '/' followed by one or more lowercase English letters.

CONSTRAINTS

- For example, "/leetcode" and "/leetcode/problems" are valid paths while an empty string and "/" are not.

Example 1

Input	`folder = ["/a","/a/b","/c/d","/c/d/e","/c/f"]`
Output	`["/a","/c/d","/c/f"]`
Explanation	*Folders "/a/b" is a subfolder of "/a" and "/c/d/e" is inside of folder "/c/d" in our filesystem.*

Example 2

Input	`folder = ["/a","/a/b/c","/a/b/d"]`
Output	`["/a"]`
Explanation	*Folders "/a/b/c" and "/a/b/d" will be removed because they are subfolders of "/a".*

Example 3

Input	`folder = ["/a/b/c","/a/b/ca","/a/b/d"]`
Output	`["/a/b/c","/a/b/ca","/a/b/d"]`

- 1 <= folder.length <= $4 * 10^4$
- 2 <= folder[i].length <= 100
- folder[i] contains only lowercase letters and '/'.
- folder[i] always starts with the character '/'.
- Each folder name is unique.

INTERVIEW APPROACH · STAR-LC

PHASE 1 CLARIFY

Let me make sure I understand the problem correctly.

Remove all folders that are sub-folders of another folder in the list.

A sub-folder has another folder's path + '/' as a prefix. Right?"

"A few quick questions:

Return the remaining folders in any order? Yes."

"Let me walk: folder=['/a','/a/b','/c/d','/c/d/e','/c/f'] -> ['/a','/c/d','/c/f'] ok

PHASE 2 BRUTE FORCE

"Let me start with brute force to lock in the logic."

BRUTE FORCE · PSEUDO-CODE

```
for folder in folders:
    check against every other folder whether it
        is inside it (prefix test with '/')
```

TIME	SPACE
O(n^2 * L)	O(1)

WHY IT IS SLOW

Compares all pairs. Sort the folders; a folder is a sub-folder if it starts with the last kept folder plus '/': O(n * L * log n).

"Should I go ahead and optimize?"

PHASE 3 OPTIMIZE

*"Sort + prefix check — O(n log n + n*L). Walk-through:"*

Sort all folder paths lexicographically.

WHY: a parent folder always comes before its sub-folders lexicographically

(e.g. '/a' < '/a/b' < '/a/b/c').

`prev = '' (last kept folder).`

For each folder in sorted order:

If folder does NOT start with prev + '/': this is NOT a sub-folder of prev -> keep it.

`result.append(folder). prev = folder.`

Else: sub-folder of prev -> skip.

WHY prev + '/': ensures we match the full path component, not just string prefix.

'/a' is NOT a sub-folder of '/ab' even though '/ab'.startswith('/a') is True.

'/a/' prefix check prevents false matches.

Time: O(n log n sort + n*L string comparison). Space: O(n).

PHASE 4 CLEAN CODE

Now I'll implement the optimized approach with meaningful names.

```python
class _1233_RemoveSubFolders:
    def removeSubfolders(self, folder):
        folder.sort()
        result = []
        prev = ''
        for f in folder:
            if not prev or not f.startswith(prev + '/'):
                result.append(f)
```

```
            prev = f
        return result
```

PHASE 5 TEST & DEBUG

"Let me trace removeSubfolders(['/a','/a/b','/c/d','/c/d/e','/c/f'])."

Sort lexicographically: ['/a','/a/b','/c/d','/c/d/e','/c/f']

result=['/a']. Check '/a/b': starts with '/a/' -> subfolder -> skip

Check '/c/d': doesn't start with '/a/' -> add. result=['/a','/c/d']

Check '/c/d/e': starts with '/c/d/' -> skip

Check '/c/f': doesn't start with '/c/d/' -> add. result=['/a','/c/d','/c/f']

ok

WHY sort + prefix check: after sorting, each folder's subfolders immediately follow it.

Check '/a/' (with trailing slash) to avoid false match like '/ab'.

PHASE 6 COMPLEXITY & FOLLOW-UP

TIME	SPACE
O(n log n + n*L)	O(n)

Follow-up: if we also need to count total files per root folder, track depth during DFS.

Follow-up: merge folders instead of removing — combine all sub-folder contents.

MEDIUM | Trie | LeetCode #1268 | 11 of 14 | 11/14

Search Suggestions System

QUESTION

You are given an array of strings products and a string searchWord.

Design a system that suggests at most three product names from products after each character of searchWord is typed. Suggested products should have common prefix with searchWord. If there are more than three products with a common prefix return the three lexicographically minimums products.

Return a list of lists of the suggested products after each character of searchWord is typed.

Example 1

Input	`products = ["mobile","mouse","moneypot","monitor","mousepad"], searchWord = "mouse"`
Output	`[["mobile","moneypot","monitor"],["mobile","moneypot","monitor"],["mouse","mousepad"],["mouse","mousepad"],["mouse","mousepad"]]`
Explanation	*products sorted lexicographically = ["mobile","moneypot","monitor","mouse","mousepad"]. After typing m and mo all products match and we show user ["mobile","moneypot","monitor"]. After typing mou, mous and mouse the system suggests ["mouse","mousepad"].*

Example 2

Input	`products = ["havana"], searchWord = "havana"`
Output	`[["havana"],["havana"],["havana"],["havana"],["havana"],["havana"]]`
Explanation	*The only word "havana" will be always suggested while typing the search word.*

CONSTRAINTS

- 1 <= products.length <= 1000
- 1 <= products[i].length <= 3000
- 1 <= sum(products[i].length) <= $2 * 10^4$
- All the strings of products are unique.
- products[i] consists of lowercase English letters.
- 1 <= searchWord.length <= 1000
- searchWord consists of lowercase English letters.

INTERVIEW APPROACH · STAR-LC

PHASE 1 CLARIFY

Let me make sure I understand the problem correctly.

Design a system that, given a search word, returns top 3 products with the same prefix after each character typed.

Products returned in lexicographic order. Right?"

"A few quick questions:

Sort products lexicographically; for each prefix find top 3 with that prefix? Yes."

"Let me walk:
products=['mobile','mouse','moneypot','monitor','mousepad'], searchWord='mouse'

-> after 'm': top3 with 'm'. after 'mo': top3 with 'mo'. etc.

PHASE 2 BRUTE FORCE

"Let me start with brute force to lock in the logic."

BRUTE FORCE · PSEUDO-CODE

```
for prefix in prefixes:
    scan all products for those starting with it,
        sort them, and take the first three
```

TIME	SPACE
O(L * n * log n)	O(n)

WHY IT IS SLOW

Rescans and re-sorts for every letter typed. Sort the products once; for each prefix binary search the first match and read the next three, or walk a trie: O(n log n + L log n).

"Should I go ahead and optimize?"

PHASE 3 OPTIMIZE

*"Build trie of products, collect suggestions at each prefix — O(n * L). Walk-through:"*

Sort products (lexicographic order).

Build trie: insert all products. At each trie node, store up to 3 products (the first 3 inserted = lexicographically smallest since sorted).

For each prefix of searchWord:

Navigate trie. At current node: return its stored list of up to 3 products.

If prefix not in trie: remaining characters return [].

WHY trie: share common prefixes -> space efficient. O(L) per query.

Alternative: binary search — sort products; for prefix of length k, binary search for first

product >= prefix and collect next 3 with the same prefix.

Time: O(n*L) build + O(m*3) query where m = len(searchWord). Space: O(n*L).

PHASE 4 CLEAN CODE

Now I'll implement the optimized approach with meaningful names.

```
import bisect
class _1268_SearchSuggestionsSystem:
    def suggestedProducts(self, products, searchWord):
        products.sort()
        result = []
        prefix = ''
        for ch in searchWord:
            prefix += ch
            start = bisect.bisect_left(products, prefix)
            suggestions = []
            for i in range(start, min(start + 3, len(produ
cts))):
                if products[i].startswith(prefix):
                    suggestions.append(products[i])
            result.append(suggestions)
        return result
```

PHASE 5 TEST & DEBUG

"Let me trace searchSuggestions(products=['mobile','mouse','moneypot','monitor','mousepad'],searchWord='mouse')."

Sort products -> ['mobile','moneypot','monitor','mouse','mousepad']

Prefix 'm': bisect_left finds start -> all 5 match 'm'. Take first 3 -> ['mobile','moneypot','monitor'] ok

Prefix 'mo': 'mobile'->no 'mo'? wait: 'mobile' starts with 'mo' ok. 'moneypot' ok, 'monitor' ok -> 3 results ok

Prefix 'mou': 'mobile'->no; 'moneypot'->no; 'monitor'->no; 'mouse'->yes; 'mousepad'->yes -> 2 results ok

Prefix 'mous': ['mouse','mousepad'] -> 2 results ok

Prefix 'mouse': ['mouse','mousepad'] -> 2 results ok

Return [['mobile','moneypot','monitor'],['moneypot','monitor','mouse'],['mouse','mousepad'],['mouse','mousepad'],['mouse','mousepad']] ok

WHY sort + binary search: O((n log n) build; O(len(word) * log n) per query to find start position.

Slide: check startswith(prefix) until prefix mismatch -> at most 3 results collected.

PHASE 6 COMPLEXITY & FOLLOW-UP

TIME	SPACE
O(n log n + m log n)	**O(1)**
sort once + m binary searches	extra

Trie approach: O(n*k) build + O(m) queries — better when m >> n.

Follow-up: if search word changes character (delete), Trie handles it more naturally.

MEDIUM | Trie | LeetCode #1698 | 12 of 14 | 12/14

Number of Distinct Substrings in a String

QUESTION

Given a string s, return the number of distinct substrings of s.

A substring of a string is obtained by deleting any number of characters (possibly zero) from the front of the string and any number (possibly zero) from the back of the string.

Example 1

Input	`s = "aabbaba"`
Output	`21`
Explanation	*The set of distinct strings is ["a","b","aa","bb","ab","ba","aab","abb","bab","bba","aba","aabb","abba","bbab","baba","aabba","abbab","bbaba","aabbab","abbaba","aabbaba"]*

Example 2

Input	`s = "abcdefg"`
Output	`28`

CONSTRAINTS

- 1 <= s.length <= 500
- s consists of lowercase English letters.
- Follow up: Can you solve this problem in O(n) time complexity?

INTERVIEW APPROACH · STAR-LC

PHASE 1 CLARIFY

Let me make sure I understand the problem correctly.

Count the number of distinct non-empty substrings of s. Right?"

"A few quick questions:

All distinct substrings regardless of length? Yes."

"Let me walk: s='aababab' -> 7 distinct substrings? Expected: 'a','b','aa','ab','ba','aab','abb','aba','bab','aaba','abab','baba','aabab','abaa','bab ab','aababab'... -> 15? Let me just say we use the approach.

PHASE 2 BRUTE FORCE

"Let me start with brute force to lock in the logic."

BRUTE FORCE · PSEUDO-CODE

```
for i in range(n):
    for j in range(i_+_1,_n):
        put s[i..j] into a set
return the set size
```

TIME	SPACE
O(n^3)	O(n^3)

PHASE 2 BRUTE FORCE

WHY IT IS SLOW

O(n^2) substrings, each hashed in O(n). A suffix array with an LCP array (distinct substrings = n(n + 1)/2 minus the sum of LCP) or a suffix automaton counts them in about O(n log n).

"Should I go ahead and optimize?"

PHASE 3 OPTIMIZE

"Suffix array + LCP array — O(n log²n). Walk-through:"

Build suffix array (SA): sorted order of all suffixes of s.

Build LCP array: LCP[i] = longest common prefix of SA[i] and SA[i-1].

Total distinct substrings = total substrings - sum(LCP).

Total substrings = n*(n+1)/2.

`sum(LCP) = duplicate substrings that are prefixes of adjacent suffixes in sorted order.`

Distinct = n*(n+1)/2 - sum(LCP[1:]).

WHY suffix array: groups all substrings by their first character; LCP tracks shared prefixes.

WHY subtract LCP: LCP[i] substrings are shared between suffix[i] and suffix[i-1] -> duplicates.

Time: O(n log²n) SA build. Space: O(n).

PHASE 4 CLEAN CODE

Now I'll implement the optimized approach with meaningful names.

```
class _1698_NumberOfDistinctSubstringsInAString:
    def countDistinct(self, s):
        n = len(s)
        root = {}
        count = 0
        for i in range(n):
            node = root
            for ch in s[i:]:
                if ch not in node:
                    node[ch] = {}
                    count += 1
                node = node[ch]
        return count
```

PHASE 5 TEST & DEBUG

"Let me trace countDistinct on s='aba' (short example)."

All substrings: 'a'(x2),'b','ab','ba','aba' -> distinct: {'a','b','ab','ba','aba'} = 5

Formula: n*(n+1)/2 - sum(LCP) where LCP from suffix array

Suffix array of 'aba': sort suffixes: 'a'(idx2),'aba'(idx0),'ba'(idx1) -> SA=[2,0,1]

LCP array: LCP(SA[0],SA[1])=LCP('a','aba')=1; LCP(SA[1],SA[2])=LCP('aba','ba')=0

Answer = n*(n+1)/2 - sum(LCP) = 3*4/2 - (1+0) = 6-1 = 5 ok

Verify: 'aba' has 5 distinct substrings {'a','b','ab','ba','aba'} ok

WHY subtract LCP sum: each LCP(i,i+1) value counts duplicates between adjacent suffixes.

PHASE 5 TEST & DEBUG

Every repeated substring was counted once too many in the n*(n+1)/2 total.

PHASE 6 COMPLEXITY & FOLLOW-UP

TIME	SPACE
O(n²)	**O(n²)**
n suffixes × average n/2 length per insertion	Trie

Suffix Array with LCP: O(n log n) time, O(n) space — more complex.

Follow-up: Distinct Substrings count using Rolling Hash — O(n log n).

Follow-up: Longest Duplicate Substring (#1062) — find the actual longest duplicate.

IN YOUR APP | Expose | Community

MEDIUM | Trie | LeetCode #2452 | 13 of 14 | 13/14

Words Within Two Edits of Dictionary

QUESTION

You are given two string arrays, queries and dictionary. All words in each array comprise of lowercase English letters and have the same length.

In one edit you can take a word from queries, and change any letter in it to any other letter. Find all words from queries that, after a maximum of two edits, equal some word from dictionary.

Return a list of all words from queries, that match with some word from dictionary after a maximum of two edits. Return the words in the same order they appear in queries.

Example 1

Input	`queries = ["word","note","ants","wood"], dictionary = ["wood","joke","moat"]`
Output	`["word","note","wood"]`
Explanation	*- Changing the 'r' in "word" to 'o' allows it to equal the dictionary word "wood". - Changing the 'n' to 'j' and the 't' to 'k' in "note" changes it to "joke". - It would take more than 2 edits for "ants" to equal a dictionary word. - "wood" can remain unchanged (0 edits) and match the corresponding dictionary word. Thus, we return ["word","note","wood"].*

Example 2

Input	`queries = ["yes"], dictionary = ["not"]`
Output	`[]`
Explanation	*Applying any two edits to "yes" cannot make it equal to "not". Thus, we return an empty array.*

CONSTRAINTS

- 1 <= queries.length, dictionary.length <= 100
- n == queries[i].length == dictionary[j].length
- 1 <= n <= 100
- All queries[i] and dictionary[j] are composed of lowercase English letters.

INTERVIEW APPROACH · STAR-LC

PHASE 1 CLARIFY

Let me make sure I understand the problem correctly.

Return all words within 2 edits (insertions, deletions, replacements) of any word in dictionary.

Right?"

"A few quick questions:

PHASE 1 CLARIFY

Use brute force edit distance check for small word lengths? Yes — words are short."

"Let me walk: queries=['word'], dictionary=['wood','good'] -> ['word'] (word->wood 1 edit, word->good 2 edits) ok

PHASE 2 BRUTE FORCE

"Let me start with brute force to lock in the logic."

BRUTE FORCE · PSEUDO-CODE

```
for word in words:
    compare it with every dictionary word letter
        by letter
    keep the query if some word differs in at
        most two positions
```

TIME	SPACE
O(q * d * L)	O(1)

NOTE

Not slow for the constraints (small lengths and counts); it is the intended approach. For larger inputs a trie that carries an edit budget prunes branches early.

"Should I go ahead and optimize?"

PHASE 3 OPTIMIZE

The bottleneck is the O(|Q|*|D|*L²) full edit distance computation.

For same-length words, edit distance <= 2 means at most 2 differing character positions.

If words have same length, count differing chars; if <= 2, within 2 edits.

Build a Trie of dictionary; for each query, DFS with edit budget.

Time: O(|D|*L + |Q|*L*26^2) with Trie. For small L, the brute is already fast."

Time: O(m) per operation — trie traversal by word length m.

PHASE 4 CLEAN CODE

Now I'll implement the optimized approach with meaningful names.

```python
class _2452_WordsWithinTwoEdits:
    def twoEditWords(self, queries, dictionary):
        def edit_within_2(w1, w2):
            if abs(len(w1) - len(w2)) > 2:
                return False
            diff = 0
            if len(w1) == len(w2):
                for a, b in zip(w1, w2):
                    if a != b:
                        diff += 1
                        if diff > 2:
                            return False
                return True
            prev = list(range(len(w2) + 1))
            for c1 in w1:
                curr = [prev[0] + 1]
```

```
                for j, c2 in enumerate(w2):
                    curr.append(min(prev[j] + (c1 != c2),
prev[j+1] + 1, curr[j] + 1))
                prev = curr
                if min(prev) > 2:
                    return False
            return prev[-1] <= 2
        return [q for q in queries if any(edit_within_2(q,
 d) for d in dictionary)]
```

PHASE 5 TEST & DEBUG

"Let me trace the trie navigation (prefix matches) and how it decides the answer."

queries=['word'], dictionary=['wood','good']: word vs wood: 1 diff->True. -> ['word'] ok

queries=['query'], dictionary=['quite','quiet']: word lengths differ -> full DP. ok

Same-length shortcut avoids full DP for the common case

Edge: query not in dictionary and no word within 2 edits -> not in result ok. Empty query -> matches empty strings only. words with different lengths need full DP edit distance.

PHASE 6 COMPLEXITY & FOLLOW-UP

TIME	SPACE
O(|Q|*|D|*L²) worst case	**O(L)**

Follow-up: Trie + DFS with edit budget — O(|D|*L + |Q|*L*26²).

Follow-up: Edit Distance (#72) — exact distance, not within 2.

IN YOUR APP | Expose | Editorial | Community

MEDIUM | Trie | LeetCode #3043 | 14 of 14 | 14/14

Find the Length of the Longest Common Prefix

QUESTION

You are given two arrays with positive integers arr1 and arr2.

A prefix of a positive integer is an integer formed by one or more of its digits, starting from its leftmost digit. For example, 123 is a prefix of the integer 12345, while 234 is not.

A common prefix of two integers a and b is an integer c, such that c is a prefix of both a and b. For example, 5655359 and 56554 have common prefixes 565 and 5655 while 1223 and 43456 do not have a common prefix.

You need to find the length of the longest common prefix between all pairs of integers (x, y) such that x belongs to arr1 and y belongs to arr2.

Return the length of the longest common prefix among all pairs. If no common prefix exists among them, return 0.

Example 1

Input	`arr1 = [1,10,100], arr2 = [1000]`
Output	`3`
Explanation	*There are 3 pairs (arr1[i], arr2[j]):* *- The longest common prefix of (1, 1000) is 1.* *- The longest common prefix of (10, 1000) is 10.* *- The longest common prefix of (100, 1000) is 100.* *The longest common prefix is 100 with a length of 3.*

Example 2

Input	`arr1 = [1,2,3], arr2 = [4,4,4]`
Output	`0`
Explanation	*There exists no common prefix for any pair (arr1[i], arr2[j]), hence we return 0.* *Note that common prefixes between elements of the same array do not count.*

CONSTRAINTS

- 1 <= arr1.length, arr2.length <= $5 * 10^4$
- 1 <= arr1[i], arr2[i] <= 10^8

INTERVIEW APPROACH · STAR-LC

PHASE 1 CLARIFY

Let me make sure I understand the problem correctly.

Find the length of the longest common prefix among all pairs (arr1[i], arr2[j])

where we convert each integer to a string. Right?"

"A few quick questions:

PHASE 1 CLARIFY

Length of the longest common prefix across all arr1,arr2 pairs? Just the max pairwise length."

"Let me walk: arr1=[1,10,100], arr2=[1000] -> '1' is prefix of all -> longest=3 (for 100 vs 1000)? Wait: 100 vs 1000 -> '100' is prefix of '1000' -> len=3 ok

PHASE 2 BRUTE FORCE

"Let me start with brute force to lock in the logic."

BRUTE FORCE · PSEUDO-CODE

```
for (a from arr1, b from arr2) in pairs(numbers):
    compare their digits from the left and keep
        the longest common prefix length
```

TIME	SPACE
O(n * m * L)	O(1)

WHY IT IS SLOW

Compares every pair. Put every prefix of every number of arr1 into a hash set, then for each number of arr2 find its longest prefix in the set: O((n + m) * L).

"Should I go ahead and optimize?"

PHASE 3 OPTIMIZE

*"Trie or prefix-set of arr1, then check arr2 prefixes — O((n+m)*L). Walk-through:"*

Step 1 — Insert all numeric prefixes of arr1 into a set:

For each number in arr1: convert to string, insert all prefixes ('1','12','123',...).

Step 2 — For each number in arr2: check all its prefixes against the set.

For each prefix length l from 1 to len(str(num)):

If str(num)[:l] in prefix_set: best = max(best, l). Break (longest prefix found).

WHY prefixes of arr1: we want the longest string that is a prefix of BOTH an arr1 number

and an arr2 number. Storing all arr1 prefixes lets us check arr2 in O(L) per number.

Trie alternative: O(L) per insertion/query, same total complexity, but cleaner code.

Time: O((n+m)*L) where L = max digits. Space: O(n*L).

PHASE 4 CLEAN CODE

Now I'll implement the optimized approach with meaningful names.

```python
class _3043_FindLengthLongestCommonPrefix:
    def longestCommonPrefix(self, arr1, arr2):
        prefix_set = set()
        for num in arr1:
            s = str(num)
```

```
            for i in range(1, len(s) + 1):
                prefix_set.add(s[:i])
        best = 0
        for num in arr2:
            s = str(num)
            for i in range(1, len(s) + 1):
                if s[:i] in prefix_set:
                    best = max(best, i)
        return best
```

PHASE 5 TEST & DEBUG

"Let me trace longestCommonPrefix(['abab','ab'],['ab','ba'])."

Use Trie: insert all strings from arr1 into trie, record depths

For each string in arr2, query longest common prefix in trie

Insert 'abab': a->b->a->b (depth 4). Insert 'ab': a->b (depth 2)

Query 'ab': traverse a->b -> prefix 'ab' exists (depth 2). Return 2 ok

Query 'ba': traverse b->a -> prefix 'ba', but trie has only 'a' at root -> prefix=0

Max found: 2 ok

WHY trie: O(total_len) build, O(len) per query; efficient for multiple prefix lookups.

PHASE 6 COMPLEXITY & FOLLOW-UP

TIME	SPACE
O((n+m)*L) n insertions of L prefixes + m lookups of L prefixes	**O(n*L)**

Trie approach: same O((n+m)*L) but explicit prefix structure.

Follow-up: Longest Common Prefix (#14) — prefix among a single list, not cross-product.